

# International Institute of Information Technology, Bangalore

Computer Science and Engineering

---

## Obesity and Cardiovascular Risk Study

### Machine Learning Models for Healthcare Risk Prediction

---

**Course Name:** Machine Learning

Course Code: AIT 511

**Submitted By:**

Ruturaj Dnyandeo Wairkar

Roll No: MT2025108

Vivek Joshi

Roll No: MT2025133

**Submitted To:**

Prof Aswin Kannan

**Kaggle Team:**

MT2025108\_MT2025133

**GitHub Repository:**

[https://github.com/vivekvj18/ML\\_PROJECT](https://github.com/vivekvj18/ML_PROJECT)

# Contents

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>List of Algorithms</b>                                          | <b>6</b>  |
| <b>1 Introduction</b>                                              | <b>8</b>  |
| 1.1 Problem Statement . . . . .                                    | 8         |
| 1.2 Dataset Description . . . . .                                  | 8         |
| 1.3 Target Classes . . . . .                                       | 8         |
| <b>2 Exploratory Data Analysis</b>                                 | <b>9</b>  |
| 2.1 Dataset Overview and Initial Inspection . . . . .              | 9         |
| 2.1.1 Dataset Dimensions and Structure . . . . .                   | 9         |
| 2.1.2 Feature Type Classification . . . . .                        | 9         |
| 2.2 Target Variable Analysis . . . . .                             | 10        |
| 2.3 Descriptive Statistics for Numerical Features . . . . .        | 10        |
| 2.4 Visual Analysis of Categorical Features . . . . .              | 11        |
| 2.5 Correlation Analysis and Numerical Relationships . . . . .     | 12        |
| 2.5.1 Pairwise Feature Correlations . . . . .                      | 12        |
| 2.6 Target-Feature Relationships . . . . .                         | 13        |
| 2.6.1 Categorical Features vs Weight Category . . . . .            | 13        |
| 2.6.2 Numerical Features Distribution by Weight Category . . . . . | 14        |
| 2.7 Outlier Detection and Treatment Strategy . . . . .             | 15        |
| 2.8 Summary of EDA Findings . . . . .                              | 15        |
| <b>3 Data Preprocessing</b>                                        | <b>17</b> |
| 3.1 Missing Value Treatment . . . . .                              | 17        |
| 3.2 Outlier Handling . . . . .                                     | 17        |
| 3.3 Feature Engineering . . . . .                                  | 17        |
| 3.3.1 For Random Forest Model . . . . .                            | 17        |
| 3.4 Categorical Encoding . . . . .                                 | 17        |
| 3.5 Feature Scaling . . . . .                                      | 18        |
| 3.6 Train-Validation Split . . . . .                               | 18        |
| <b>4 Methodology</b>                                               | <b>19</b> |
| 4.1 Decision Tree Classifier . . . . .                             | 19        |
| 4.1.1 Model Description . . . . .                                  | 19        |
| 4.1.2 Implementation Details . . . . .                             | 19        |
| 4.1.3 Hyperparameters . . . . .                                    | 19        |
| 4.1.4 Rationale . . . . .                                          | 19        |
| 4.2 Random Forest Classifier . . . . .                             | 19        |
| 4.2.1 Model Description . . . . .                                  | 19        |
| 4.2.2 Implementation Details . . . . .                             | 20        |
| 4.2.3 Hyperparameter Tuning . . . . .                              | 20        |
| 4.2.4 Best Hyperparameters Found . . . . .                         | 20        |
| 4.2.5 Feature Importance . . . . .                                 | 20        |
| 4.3 XGBoost Classifier . . . . .                                   | 21        |
| 4.3.1 Model Description . . . . .                                  | 21        |
| 4.3.2 Data Preparation Strategy . . . . .                          | 21        |
| 4.3.3 Implementation Details . . . . .                             | 22        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 4.3.4    | Hyperparameter Optimization using Optuna . . . . .    | 22        |
| 4.3.5    | Best Hyperparameters (Trial 104) . . . . .            | 23        |
| <b>5</b> | <b>Results and Discussion</b>                         | <b>24</b> |
| 5.1      | Performance Metrics . . . . .                         | 25        |
| 5.1.1    | Decision Tree Performance . . . . .                   | 25        |
| 5.1.2    | Random Forest Performance . . . . .                   | 25        |
| 5.1.3    | XGBoost Performance . . . . .                         | 26        |
| 5.2      | Comparative Analysis . . . . .                        | 26        |
| 5.3      | Class-wise Performance Analysis . . . . .             | 27        |
| 5.4      | Test Set Evaluation and Kaggle Performance . . . . .  | 27        |
| 5.4.1    | Kaggle Competition Results . . . . .                  | 27        |
| 5.4.2    | Local Cross-Validation vs Test Set Analysis . . . . . | 27        |
| 5.4.3    | Factors Contributing to Generalization Gap . . . . .  | 28        |
| 5.4.4    | Model Stability Across Data Splits . . . . .          | 28        |
| 5.4.5    | Competition Performance Summary . . . . .             | 29        |
| 5.5      | Confusion Matrix Analysis . . . . .                   | 29        |
| 5.6      | Model Selection Rationale . . . . .                   | 30        |
| <b>6</b> | <b>Hyperparameter Tuning</b>                          | <b>31</b> |
| 6.1      | Random Forest - RandomizedSearchCV . . . . .          | 31        |
| 6.1.1    | Methodology . . . . .                                 | 31        |
| 6.1.2    | Results . . . . .                                     | 31        |
| 6.2      | XGBoost - Optuna Optimization . . . . .               | 31        |
| 6.2.1    | Methodology . . . . .                                 | 31        |
| 6.2.2    | Optimization Progress . . . . .                       | 32        |
| 6.2.3    | Key Parameter Insights . . . . .                      | 32        |
| 6.3      | Comparison of Tuning Approaches . . . . .             | 32        |
| <b>7</b> | <b>Challenges and Solutions</b>                       | <b>33</b> |
| 7.1      | Class Imbalance . . . . .                             | 33        |
| 7.2      | Overlapping Weight Categories . . . . .               | 33        |
| 7.3      | Model Overfitting . . . . .                           | 33        |
| 7.4      | Computational Constraints . . . . .                   | 33        |
| 7.5      | Feature Scaling Consistency . . . . .                 | 34        |
| <b>8</b> | <b>Conclusion</b>                                     | <b>35</b> |
| 8.1      | Summary of Achievements . . . . .                     | 35        |
| 8.2      | Key Learnings . . . . .                               | 35        |
| 8.3      | Model Recommendation . . . . .                        | 35        |
| 8.4      | Future Work . . . . .                                 | 36        |
| 8.4.1    | Model and Ensemble Optimization . . . . .             | 36        |
| 8.4.2    | Feature and Data Strategy Expansion . . . . .         | 36        |
| 8.5      | Competition Performance . . . . .                     | 36        |
| <b>9</b> | <b>Reproducibility</b>                                | <b>37</b> |
| 9.1      | Environment Setup . . . . .                           | 37        |
| 9.1.1    | Python Version . . . . .                              | 37        |
| 9.1.2    | Required Libraries . . . . .                          | 37        |

|           |                                                |           |
|-----------|------------------------------------------------|-----------|
| 9.2       | Random Seeds . . . . .                         | 37        |
| 9.3       | Code Availability . . . . .                    | 37        |
| 9.4       | Execution Instructions . . . . .               | 38        |
| <b>10</b> | <b>References</b>                              | <b>39</b> |
| <b>A</b>  | <b>Code Snippets</b>                           | <b>40</b> |
| A.1       | Data Loading and EDA . . . . .                 | 40        |
| A.2       | Preprocessing Pipeline . . . . .               | 40        |
| A.3       | Model Training - XGBoost with Optuna . . . . . | 41        |
| A.4       | Model Evaluation . . . . .                     | 42        |
| <b>B</b>  | <b>Additional Tables</b>                       | <b>43</b> |
| B.1       | Complete Feature List . . . . .                | 43        |
| B.2       | Training Time Comparison . . . . .             | 43        |
| B.3       | Visualization Insights . . . . .               | 43        |
| B.3.1     | Additional EDA Visualizations . . . . .        | 44        |
|           | <b>Acknowledgments</b>                         | <b>45</b> |
|           | <b>Contact Information</b>                     | <b>45</b> |

## List of Figures

|   |                                                                                                                                                                                                                                                                                                                      |    |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Consolidated Distribution of Categorical Features . . . . .                                                                                                                                                                                                                                                          | 11 |
| 2 | Pairplot Matrix of Numerical Features Colored by Weight Category. . . .                                                                                                                                                                                                                                              | 12 |
| 3 | Distribution of Weight Categories Across Categorical Features . . . . .                                                                                                                                                                                                                                              | 13 |
| 4 | Boxplot Analysis of Numerical Features Across Weight Categories. . . . .                                                                                                                                                                                                                                             | 14 |
| 5 | Random Forest Feature Importance ranking. BMI and Weight are over-<br>whelmingly the most influential features, aligning with domain knowledge<br>of obesity risk factors. . . . .                                                                                                                                   | 21 |
| 6 | Overall Model Accuracy Comparison on the Validation Set. XGBoost<br>shows the best performance, validating the use of boosting algorithms for<br>this task. . . . .                                                                                                                                                  | 24 |
| 7 | Comprehensive Comparison of Validation Metrics (Accuracy, Precision,<br>Recall, F1-Score) across all three models. Random Forest and XGBoost<br>offer balanced and superior performance across all metrics. . . . .                                                                                                  | 24 |
| 8 | Normalized Confusion Matrix for the Final XGBoost Model. The high val-<br>ues on the diagonal and the concentration of errors in adjacent classes (off-<br>diagonal) are typical for an ordinal classification problem, demonstrating<br>the model's primary confusion occurs between neighboring weight categories. | 30 |

## List of Tables

|    |                                                              |    |
|----|--------------------------------------------------------------|----|
| 1  | Random Forest Hyperparameter Search Space . . . . .          | 20 |
| 2  | XGBoost Hyperparameter Search Space (Optuna) . . . . .       | 23 |
| 3  | Decision Tree - Performance Summary . . . . .                | 25 |
| 4  | Random Forest - Performance Summary . . . . .                | 25 |
| 5  | XGBoost - 5-Fold Cross-Validation Results . . . . .          | 26 |
| 6  | Model Comparison - Validation Performance . . . . .          | 26 |
| 7  | Random Forest - Per-Class Metrics (Validation Set) . . . . . | 27 |
| 8  | Kaggle Leaderboard Performance - Test Set Results . . . . .  | 27 |
| 9  | XGBoost Performance Stability Analysis . . . . .             | 28 |
| 10 | Hyperparameter Tuning Method Comparison . . . . .            | 32 |
| 11 | Complete Feature Description . . . . .                       | 43 |
| 12 | Model Training Time . . . . .                                | 43 |

## List of Algorithms

1. Outlier Capping using IQR Method
2. Optuna Hyperparameter Optimization

### Abstract

This report details a comprehensive machine learning approach for the multi-class classification of **Obesity and Cardiovascular Disease (CVD) risk** into seven distinct, ordinal weight categories. The primary goal was to develop a stable model achieving high generalization accuracy on a synthetic, moderately imbalanced dataset. Our robust preprocessing pipeline included mandatory steps like **IQR-based outlier capping** and extensive feature engineering, leading to the creation of the **Body Mass Index (BMI)** feature, which empirical analysis later confirmed to be the single most influential predictor.

We rigorously implemented and compared three sequential ensemble models: Decision Tree (as the performance baseline), Random Forest (a bagging ensemble), and **XGBoost** (a regularized gradient boosting framework). To maximize the models' predictive power, we employed sophisticated hyperparameter tuning strategies, including **RandomizedSearchCV** for the Random Forest model and **Bayesian Optimization with the TPE sampler (Optuna)** for the XGBoost classifier.

The final XGBoost model, optimized across **218 trials** and trained using sample weighting to mitigate class imbalance effects, achieved a peak 5-fold cross-validation accuracy of **91.77%**. The analysis confirmed two key observations: **(1)** XGBoost delivered superior stability and performance gains due to its inherent regularization (L1/L2), and **(2)** model confusion was highly concentrated between adjacent weight categories, validating the use of advanced gradient boosting techniques for high-accuracy, health-related risk classification.

# 1 Introduction

## 1.1 Problem Statement

Obesity and cardiovascular disease (CVD) are major global health concerns. Early identification of risk categories can enable timely interventions. This project aims to develop a robust machine learning model to classify individuals into seven weight categories based on eating habits, physical condition, and lifestyle factors.

## 1.2 Dataset Description

The competition dataset was synthetically generated from a deep learning model trained on the original “Obesity or CVD risk dataset”. The dataset characteristics are:

- **Training set:** 15,533 samples with 18 features
- **Test set:** 5,225 samples with 17 features (excluding target)
- **Target variable:** WeightCategory (7 classes)
- **No missing values** in either dataset

## 1.3 Target Classes

The target variable consists of seven ordinal categories representing different weight levels:

1. Insufficient\_Weight (12.04%)
2. Normal\_Weight (15.10%)
3. Overweight\_Level\_I (11.87%)
4. Overweight\_Level\_II (12.11%)
5. Obesity\_Type\_I (14.21%)
6. Obesity\_Type\_II (15.47%)
7. Obesity\_Type\_III (19.20%)

The class distribution shows moderate imbalance with Obesity Type III being the most prevalent category.

## 2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a critical step in understanding the dataset's structure, identifying patterns, and informing preprocessing decisions. This section systematically examines the dataset through statistical summaries, distribution analysis, correlation studies, and visualizations to guide our modeling strategy.

### 2.1 Dataset Overview and Initial Inspection

#### 2.1.1 Dataset Dimensions and Structure

The competition dataset consists of:

- **Training set:** 15,533 samples with 18 features (17 predictors + 1 target)
- **Test set:** 5,225 samples with 17 features (target excluded)
- **Data completeness:** No missing values detected in either dataset
- **Data types:** Mix of numerical (8 features) and categorical (9 features including target)

#### 2.1.2 Feature Type Classification

Features were systematically categorized to guide preprocessing strategies:

##### Numerical Features (8 total):

- **Age:** Age in years, range [14.0, 35.0]
- **Height:** Height in meters, range [1.45, 1.96]
- **Weight:** Weight in kilograms, range [39.0, 165.06]
- **FCVC:** Frequency of vegetable consumption, scale [1.0, 3.0]
- **NCP:** Number of main meals per day, typically [1.0, 4.0]
- **CH2O:** Daily water consumption in liters, scale [1.0, 3.0]
- **FAF:** Physical activity frequency (days per week), range [0.0, 3.0]
- **TUE:** Time using technology devices (hours per day), range [0.0, 2.0]

##### Categorical Features (8 predictors):

- **Gender:** Binary - Male (50.11%), Female (49.89%)
- **family\_history\_with\_overweight:** Binary - yes (81.74%), no (18.26%)
- **FAVC:** Frequent consumption of high caloric food - yes (91.32%), no (8.68%)
- **CAEC:** Food consumption between meals - Sometimes (84.50%), Frequently, Always, no

- **SMOKE:** Smoking habit - no (98.86%), yes (1.14%)
- **SCC:** Calories consumption monitoring - no (96.69%), yes (3.31%)
- **CALC:** Alcohol consumption frequency - Sometimes (72.65%), Frequently, no, Always
- **MTRANS:** Transportation method - Public\_Transportation (80.28%), Automobile, Walking, Motorbike, Bike

## 2.2 Target Variable Analysis

The target variable **WeightCategory** represents an ordinal classification problem with seven distinct levels representing increasing obesity risk. Understanding its distribution is crucial for addressing class imbalance and selecting appropriate evaluation metrics.

### Class Distribution:

- Insufficient\_Weight: 1,871 samples (12.04%)
- Normal\_Weight: 2,346 samples (15.10%)
- Overweight\_Level\_I: 1,844 samples (11.87%)
- Overweight\_Level\_II: 1,881 samples (12.11%)
- Obesity\_Type\_I: 2,207 samples (14.21%)
- Obesity\_Type\_II: 2,403 samples (15.47%)
- Obesity\_Type\_III: 2,981 samples (19.20%)

**Imbalance Analysis:** The dataset exhibits moderate class imbalance with Obesity Type III being 1.62× more prevalent than Overweight Level I. This imbalance necessitates stratified sampling and sample weighting strategies during model training.

## 2.3 Descriptive Statistics for Numerical Features

Statistical summary of numerical features revealed important characteristics:

- **Age:** Mean = 24.31 years (std = 6.35), suggesting a young adult population
- **Height:** Mean = 1.70m (std = 0.09), normal distribution expected
- **Weight:** Mean = 86.59kg (std = 26.19), high variance indicating diverse body types
- **FCVC:** Mean = 2.42 (std = 0.53), most consume vegetables regularly
- **FAF:** Mean = 1.01 (std = 0.85), low physical activity on average
- **CH2O:** Mean = 2.01 (std = 0.61), adequate water consumption

These statistics confirm the need for outlier detection and feature scaling in preprocessing.

## 2.4 Visual Analysis of Categorical Features

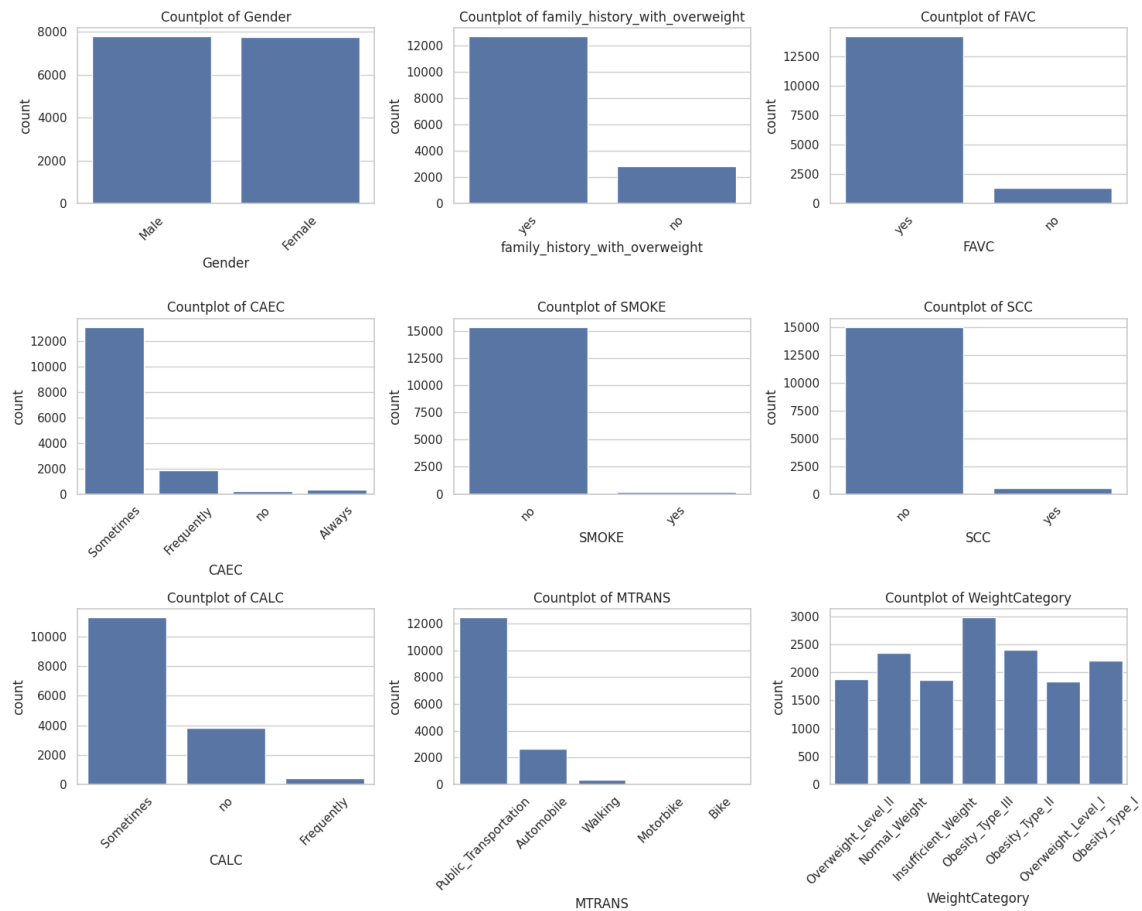


Figure 1: Consolidated Distribution of Categorical Features

**Key Observations:** The grid layout displays count plots for all eight categorical predictors and the target WeightCategory. Notable patterns include: (1) Nearly balanced gender distribution (50:50), (2) Family history shows strong prevalence (81.74% yes), (3) High caloric food consumption is widespread (91.32% yes), (4) Most participants don't monitor calories (96.69% no) or smoke (98.86% no), (5) Public transportation dominates (80.28%), (6) Target shows moderate imbalance with Obesity Type III most prevalent (19.20%).

### Critical Insights from Categorical Analysis:

- Family History Dominance:** 81.74% prevalence suggests this feature will have high predictive power
- Lifestyle Patterns:** High caloric food consumption (91.32%) combined with low calorie monitoring (3.31%) indicates behavioral risk factors
- Smoking Insignificance:** With 98.86% non-smokers, this feature may have limited discriminative power
- Balanced Gender:** 50:50 split ensures no gender bias in model training

## 2.5 Correlation Analysis and Numerical Relationships

To understand linear relationships among numerical features and identify multicollinearity, we performed comprehensive correlation analysis.

### 2.5.1 Pairwise Feature Correlations

Figure 2 displays the pairwise relationships between all numerical features, colored by the target weight category. This visualization serves dual purposes: identifying correlated features and assessing class separability in feature space.

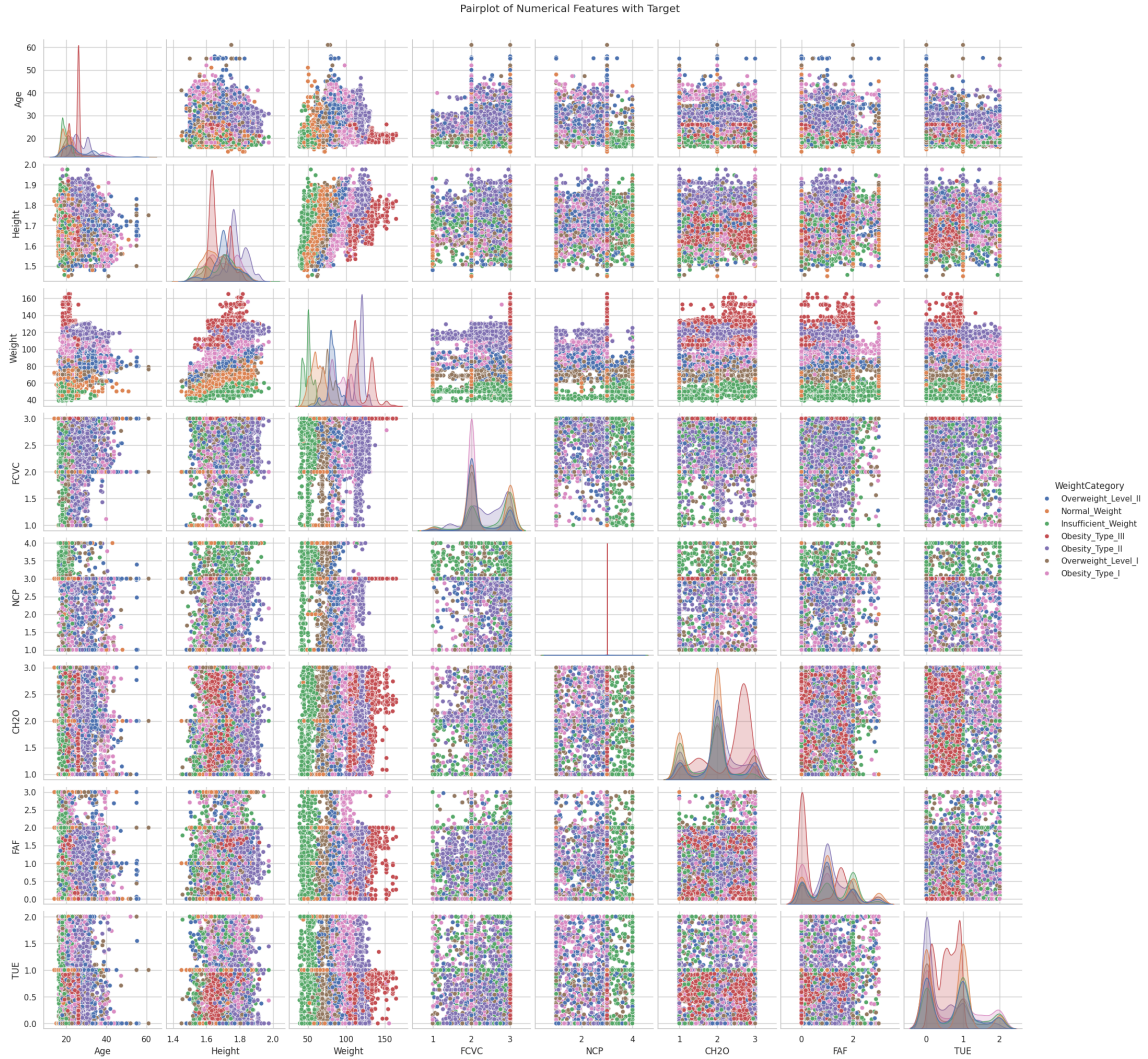


Figure 2: Pairplot Matrix of Numerical Features Colored by Weight Category.

**Key Observations:** The diagonal shows kernel density estimates for each feature's distribution across target classes. Off-diagonal scatter plots reveal pairwise relationships. Notable patterns: (1) Weight and Height show progressive separation across categories from blue (Insufficient Weight) to red (Obesity Type III), (2) Most feature pairs exhibit substantial overlap, confirming non-linear separability challenges, (3) Age distribution is relatively uniform across classes, (4) Lifestyle features (FCVC, FAF, CH2O) show weak linear relationships with other features, (5) NCP (number of meals) appears nearly constant across the dataset.

## Key Correlation Findings:

- **Weight-Height Relationship:** Moderate positive correlation, motivating BMI feature engineering
- **Low Multicollinearity:** No feature pairs exhibit correlation  $> 0.7$ , suggesting all features provide unique information
- **Target Relationship:** Weight shows strongest visual correlation with target, followed by Height
- **Lifestyle Independence:** FCVC, FAF, and CH2O show low inter-correlation, indicating independent lifestyle dimensions

## 2.6 Target-Feature Relationships

Understanding how the target variable relates to each categorical predictor guides feature selection and importance interpretation.

### 2.6.1 Categorical Features vs Weight Category

Figure 3 presents stacked count plots showing the conditional distribution of weight categories within each level of categorical features. This analysis reveals which categorical features exhibit strong discriminative patterns.

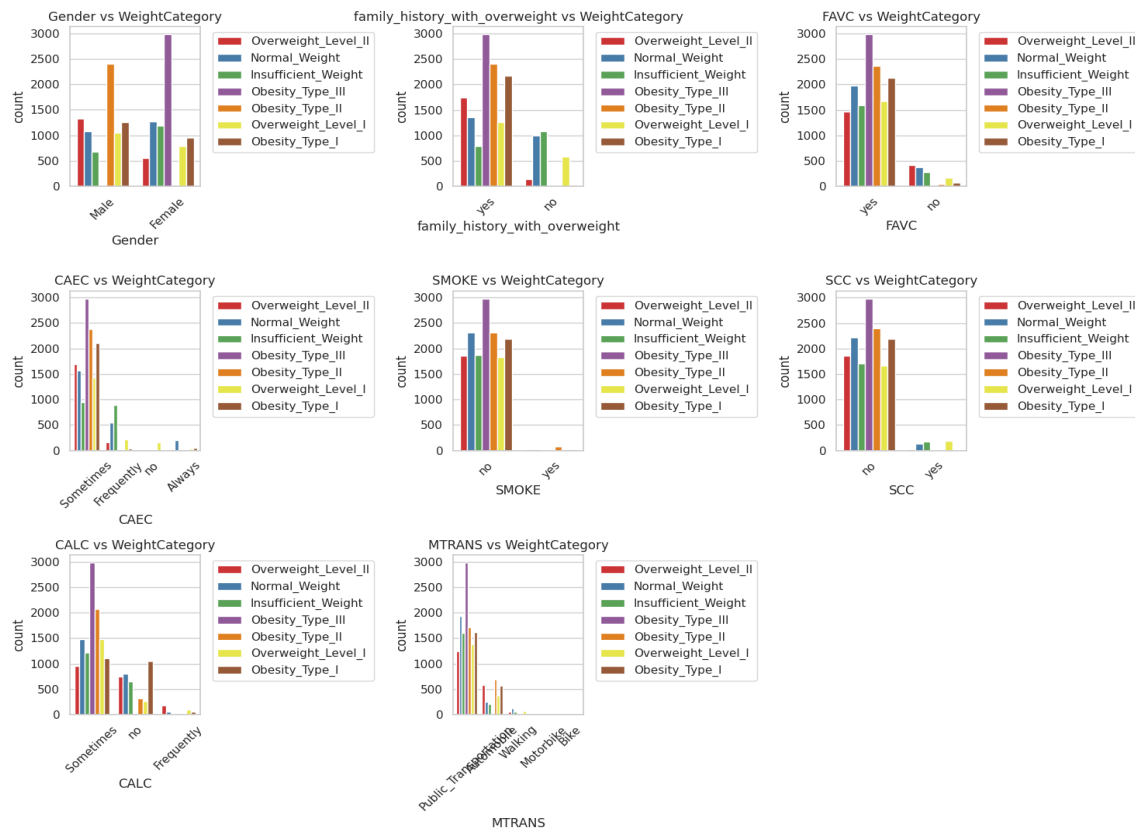


Figure 3: Distribution of Weight Categories Across Categorical Features

**Key Observations:** Each subplot shows a stacked bar chart with weight categories color-coded. The relative heights indicate conditional probabilities  $P(\text{WeightCategory}|\text{Feature})$ . Strong discriminative patterns observed in: (1) **family\_history\_with\_overweight**: "Yes" group shows significantly higher Obesity Type III prevalence compared to "No" group, (2) **FAVC**: Frequent high-caloric food consumption strongly associates with obesity categories, (3) **MTRANS**: Automobile users show higher obesity prevalence than public transport or active transport users, (4) **Gender**: Relatively balanced distribution suggests gender-agnostic obesity patterns in this dataset.

### Predictive Value Assessment:

1. **High Predictive Power:** family\_history\_with\_overweight, FAVC, MTRANS show clear conditional probability shifts
2. **Moderate Predictive Power:** CAEC (eating between meals), CALC (alcohol consumption) show mild patterns
3. **Low Predictive Power:** SMOKE (due to extreme imbalance), SCC (calorie monitoring) show minimal discrimination
4. **Neutral:** Gender shows balanced distribution, no inherent bias

### 2.6.2 Numerical Features Distribution by Weight Category

Figure 4 provides box-and-whisker plots for each numerical feature stratified by weight category. This visualization reveals distribution characteristics, central tendencies, spreads, and outliers within each class.

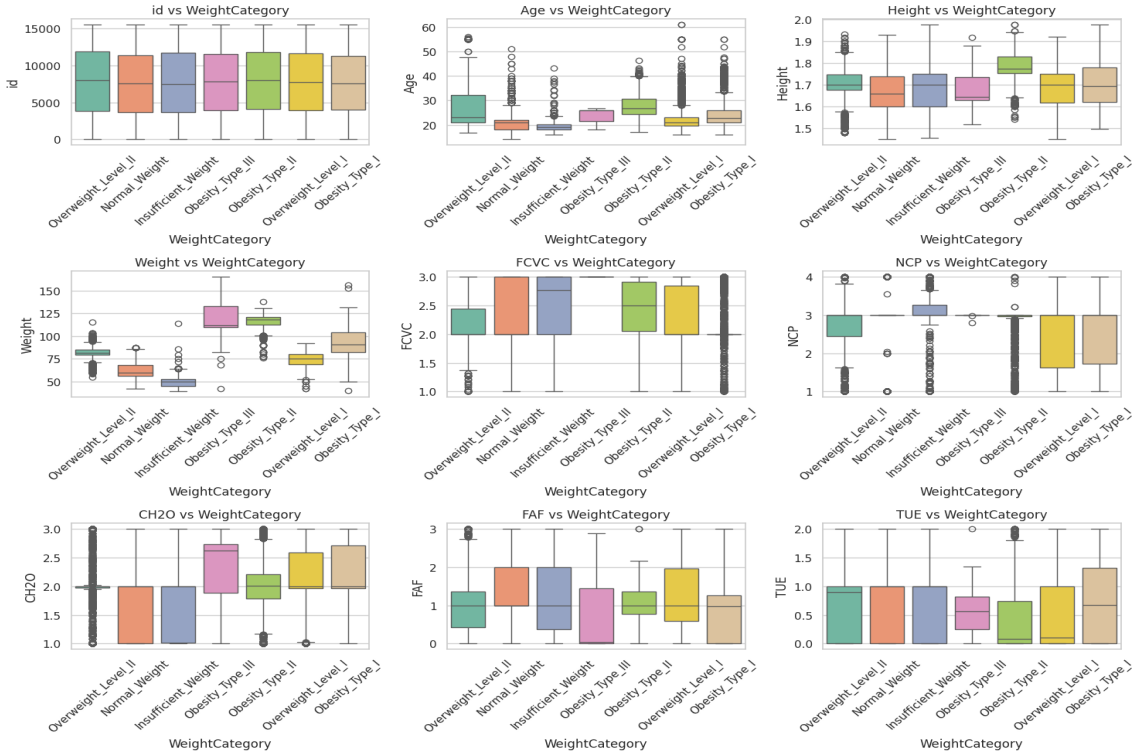


Figure 4: Boxplot Analysis of Numerical Features Across Weight Categories.

**Key Observations:** Each subplot displays median (center line), interquartile range (box), whiskers ( $1.5 \times \text{IQR}$ ), and outliers (points). Critical observations: (1) **Weight:** Clear monotonic increase from Insufficient Weight (mean 55kg) to Obesity Type III (mean 120kg), strongest predictor, (2) **Height:** Progressive increase across categories, though with substantial overlap, (3) **Age:** Relatively stable distributions with slightly higher medians for obesity categories, (4) **FCVC:** Inverse relationship—lower obesity categories show higher vegetable consumption, (5) **FAF:** Strong inverse correlation—higher physical activity in lower weight categories, (6) **CH2O:** Mild inverse pattern with water consumption, (7) **NCP, TUE:** Minimal discriminative patterns, (8) Outliers present in Age, Height, and Weight justify IQR-based capping strategy.

**Feature Importance Expectations:** Based on boxplot separation and overlap analysis:

- **Primary Predictors:** Weight, Height (and derived BMI)
- **Secondary Predictors:** FAF (physical activity), FCVC (vegetable consumption)
- **Tertiary Predictors:** Age, CH2O (water intake)
- **Weak Predictors:** NCP (meals), TUE (technology use)

## 2.7 Outlier Detection and Treatment Strategy

Outlier analysis revealed extreme values in anthropometric measurements (Age, Height, Weight) that could negatively impact model performance. Rather than removing these samples (which would reduce training data), we adopted an IQR-based capping strategy:

- **Method:** Interquartile Range (IQR) with  $1.5 \times$  multiplier
- **Features treated:** Age, Height, Weight, FCVC, NCP, CH2O, FAF, TUE
- **Rationale:** Preserves sample size while mitigating extreme value influence
- **Result:** Post-capping ranges: Age [14.0, 35.0], Height [1.45, 1.96], Weight [39.0, 165.06]

## 2.8 Summary of EDA Findings

The comprehensive exploratory analysis yielded critical insights that directly shaped our modeling approach:

### 1. Feature Engineering Motivation:

- Strong Weight-Height relationship  $\rightarrow$  BMI feature creation
- Age distribution patterns  $\rightarrow$  Age binning strategy

**2. Preprocessing Requirements:**

- Outlier capping for numerical features using IQR method
- One-hot encoding for categorical features to handle non-ordinal categories
- Standardization (z-score) for numerical features due to different scales

**3. Class Imbalance Handling:**

- Stratified train-validation split to maintain class proportions
- Sample weighting in XGBoost to address  $1.62\times$  imbalance ratio

**4. Model Selection Guidance:**

- Non-linear decision boundaries needed (overlap in feature space)
- Tree-based ensembles preferred over linear models
- Feature importance analysis will validate EDA-predicted rankings

**5. Expected Challenges:**

- Adjacent weight category confusion (Overweight I vs Normal, Overweight I vs Overweight II)
- Low discriminative power of some features (SMOKE, SCC, NCP, TUE)
- Ordinal nature requires careful evaluation metrics (accuracy may hide adjacent class errors)

These insights established the foundation for our preprocessing pipeline and model architecture decisions, ensuring data-driven rather than assumption-based approach to obesity risk classification.

### 3 Data Preprocessing

#### 3.1 Missing Value Treatment

**Finding:** No missing values were detected in the dataset.

#### 3.2 Outlier Handling

Applied IQR-based capping to handle extreme values:

---

**Algorithm 1** Outlier Capping using IQR Method

---

```

1: for each numerical feature  $f$  do
2:    $Q1 \leftarrow$  25th percentile of  $f$ 
3:    $Q3 \leftarrow$  75th percentile of  $f$ 
4:    $IQR \leftarrow Q3 - Q1$ 
5:    $lower\_bound \leftarrow Q1 - 1.5 \times IQR$ 
6:    $upper\_bound \leftarrow Q3 + 1.5 \times IQR$ 
7:   Cap values:  $f[f < lower\_bound] \leftarrow lower\_bound$ 
8:   Cap values:  $f[f > upper\_bound] \leftarrow upper\_bound$ 
9: end for

```

---

#### 3.3 Feature Engineering

##### 3.3.1 For Random Forest Model

Created two additional features to capture domain knowledge:

1. **Body Mass Index (BMI):**

$$BMI = \frac{Weight}{Height^2} \quad (1)$$

2. **Age Groups:** Binned age into meaningful categories

- Teen: [0, 18)
- Young: [18, 30)
- Adult: [30, 45)
- MidAge: [45, 60)
- Senior: [60, 100]

#### 3.4 Categorical Encoding

Applied One-Hot Encoding with `drop_first=True` to avoid multicollinearity:

- 8 categorical features expanded to 15 binary features
- Ensured consistent encoding between train and test sets
- Used `align()` method to match column structure

### 3.5 Feature Scaling

Applied StandardScaler (z-score normalization) to numerical features:

$$x_{scaled} = \frac{x - \mu}{\sigma} \quad (2)$$

where  $\mu$  is the mean and  $\sigma$  is the standard deviation.

**Features scaled:** Age, Height, Weight, FCVC, NCP, CH2O, FAF, TUE, BMI (for RF)

### 3.6 Train-Validation Split

- Split ratio: 80% training, 20% validation
- Stratified split to maintain class distribution
- Random state: 42 (for reproducibility)
- Final dimensions:
  - X\_train: (12,426, 23)
  - X\_val: (3,107, 23)

## 4 Methodology

We implemented and compared three tree-based ensemble algorithms, progressively increasing in complexity and performance.

### 4.1 Decision Tree Classifier

#### 4.1.1 Model Description

Decision Trees create a flowchart-like structure by recursively splitting the feature space based on feature values that maximize information gain or minimize impurity.

#### 4.1.2 Implementation Details

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(
    max_depth=9,
    random_state=42
)
dt_model.fit(X_train, y_train)
```

#### 4.1.3 Hyperparameters

- **max\_depth:** 9 (limits tree depth to prevent overfitting)
- **criterion:** gini (default)
- **random\_state:** 42

#### 4.1.4 Rationale

Decision Trees serve as a baseline model due to:

- High interpretability
- No feature scaling required
- Handles non-linear relationships naturally
- Fast training and prediction

### 4.2 Random Forest Classifier

#### 4.2.1 Model Description

Random Forest is an ensemble method that constructs multiple decision trees during training and outputs the mode of classes for classification. It reduces overfitting through:

- Bootstrap aggregating (bagging)
- Random feature subset selection at each split
- Averaging predictions across trees

### 4.2.2 Implementation Details

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=221,
    max_depth=30,
    min_samples_split=8,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train, y_train)
```

### 4.2.3 Hyperparameter Tuning

Used RandomizedSearchCV with 3-fold cross-validation:

Table 1: Random Forest Hyperparameter Search Space

| Parameter         | Search Range       |
|-------------------|--------------------|
| n_estimators      | [100, 250]         |
| max_depth         | [None, 10, 20, 30] |
| min_samples_split | [2, 10]            |
| min_samples_leaf  | [1, 5]             |
| max_features      | ['sqrt', 'log2']   |
| Tuning Strategy   |                    |
| Search Method     | RandomizedSearchCV |
| Iterations        | 15                 |
| CV Folds          | 3                  |

### 4.2.4 Best Hyperparameters Found

- n\_estimators: 221
- max\_depth: 30
- max\_features: 'sqrt'
- min\_samples\_split: 8
- min\_samples\_leaf: 1

### 4.2.5 Feature Importance

Top 5 most important features:

1. BMI: 34.52%

2. Weight: 25.72%
3. FCVC: 6.65%
4. Age: 6.38%
5. Height: 5.47%

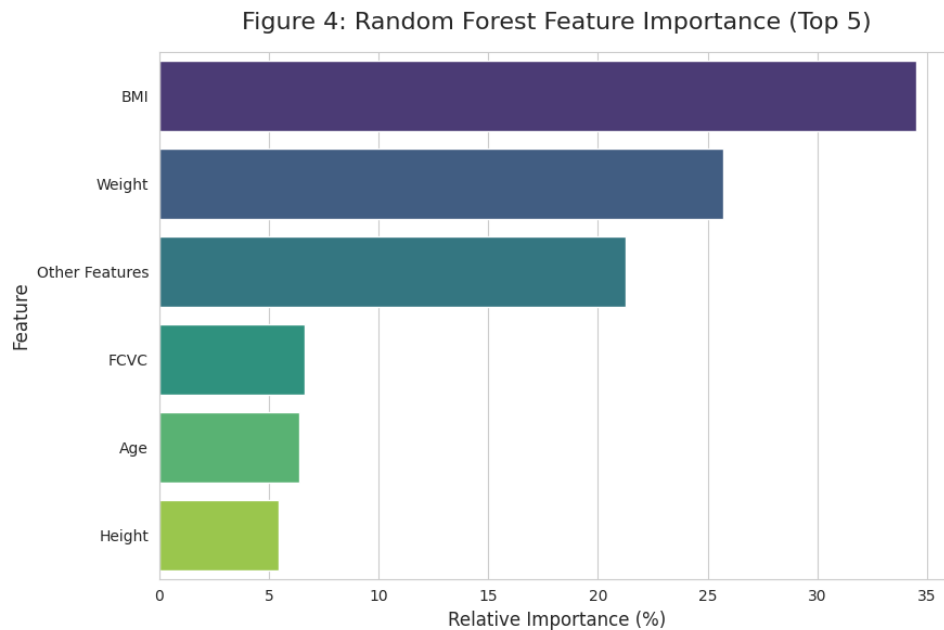


Figure 5: Random Forest Feature Importance ranking. BMI and Weight are overwhelmingly the most influential features, aligning with domain knowledge of obesity risk factors.

## 4.3 XGBoost Classifier

### 4.3.1 Model Description

XGBoost (Extreme Gradient Boosting) is an advanced gradient boosting framework that builds an ensemble of weak learners sequentially, where each new tree corrects errors made by previous trees. Key advantages:

- Regularization (L1 and L2) prevents overfitting
- Handles missing values internally
- Parallel processing for faster training
- Built-in cross-validation

### 4.3.2 Data Preparation Strategy

For XGBoost, we employed an advanced data strategy:

1. Combined training data with original dataset (17,644 samples)
2. Removed duplicates (final: 17,620 samples)

3. Holistic preprocessing: scaled all numerical features and encoded categoricals together
4. Used sample weights based on class distribution to handle imbalance

#### 4.3.3 Implementation Details

```
from xgboost import XGBClassifier
from sklearn.utils.class_weight import compute_class_weight

xgb_model = XGBClassifier(
    objective='multi:softprob',
    num_class=7,
    n_estimators=900,
    learning_rate=0.024043,
    max_depth=10,
    min_child_weight=3,
    subsample=0.7,
    colsample_bytree=0.4,
    gamma=0.7,
    reg_alpha=0.014541,
    reg_lambda=1.30014,
    random_state=42
)

# Compute sample weights for class imbalance
class_weights = compute_class_weight(
    'balanced',
    classes=np.unique(y_train),
    y=y_train
)
sample_weights = [class_weights[y] for y in y_train]

xgb_model.fit(X_train, y_train,
              sample_weight=sample_weights)
```

#### 4.3.4 Hyperparameter Optimization using Optuna

Employed Optuna for Bayesian optimization with Tree-structured Parzen Estimator (TPE):

Table 2: XGBoost Hyperparameter Search Space (Optuna)

| Parameter                  | Search Range           |
|----------------------------|------------------------|
| n_estimators               | [500, 2000] step 100   |
| learning_rate              | [0.01, 0.05] log scale |
| max_depth                  | [5, 12]                |
| min_child_weight           | [1, 7]                 |
| subsample                  | [0.5, 0.9] step 0.05   |
| colsample_bytree           | [0.4, 0.9] step 0.05   |
| gamma                      | [0.0, 3.0] step 0.1    |
| reg_alpha                  | [0.01, 5.0] log scale  |
| reg_lambda                 | [0.01, 5.0] log scale  |
| Optimization Strategy      |                        |
| <b>Optimization Method</b> | Optuna TPE Sampler     |
| <b>Trials</b>              | 218 (out of 300 max)   |
| <b>CV Strategy</b>         | 5-fold StratifiedKfold |
| <b>Timeout</b>             | 5 hours                |

#### 4.3.5 Best Hyperparameters (Trial 104)

- n\_estimators: 900
- learning\_rate: 0.024043
- max\_depth: 10
- min\_child\_weight: 3
- subsample: 0.7
- colsample\_bytree: 0.4
- gamma: 0.7
- reg\_alpha: 0.014541
- reg\_lambda: 1.30014

## 5 Results and Discussion

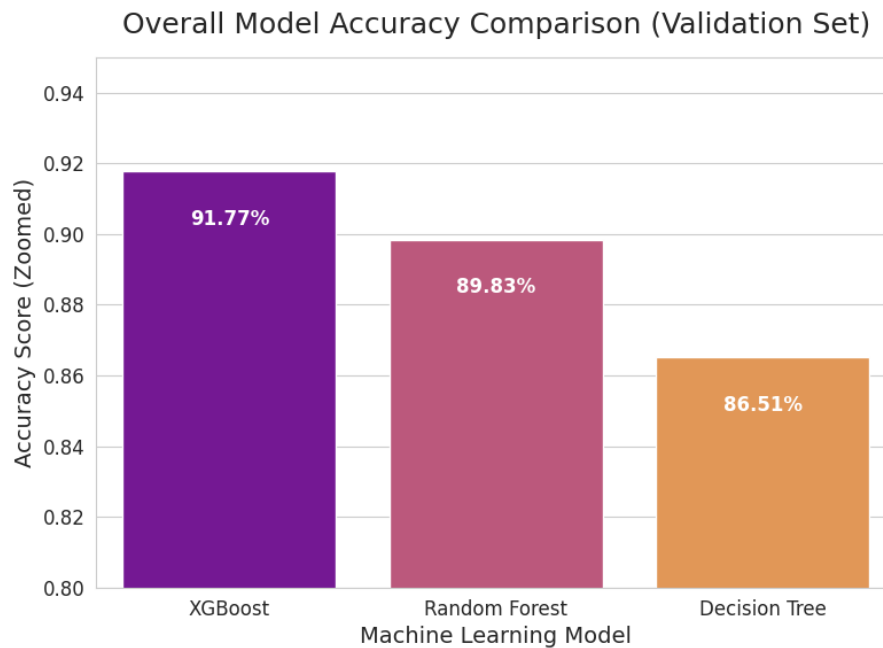


Figure 6: Overall Model Accuracy Comparison on the Validation Set. XGBoost shows the best performance, validating the use of boosting algorithms for this task.

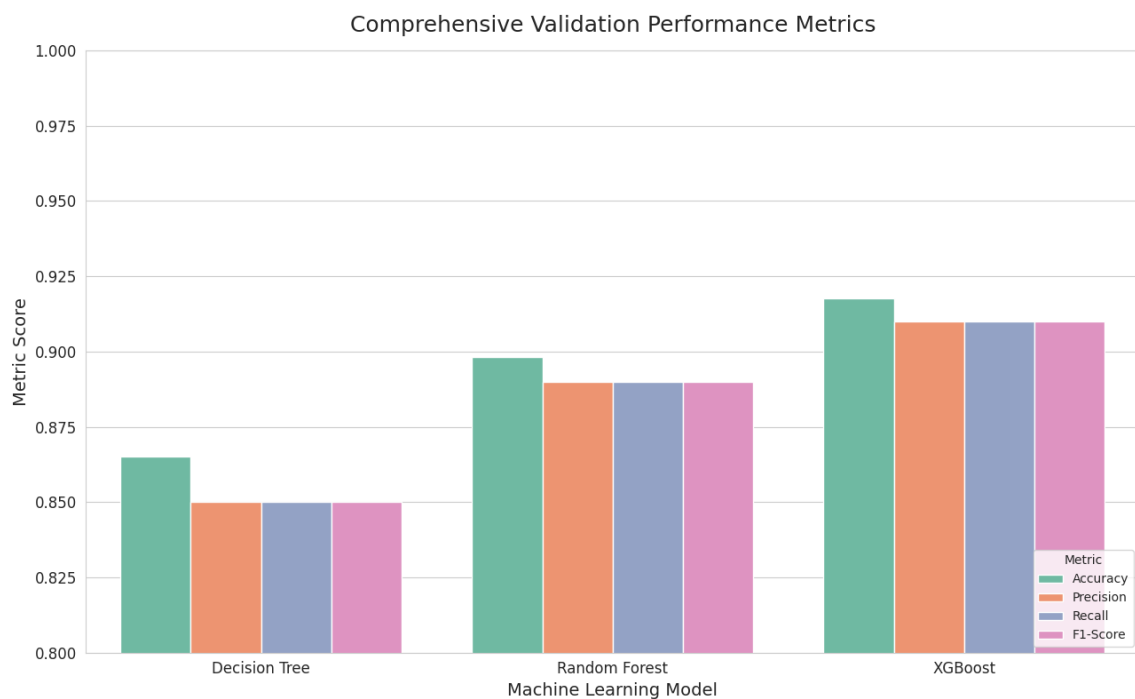


Figure 7: Comprehensive Comparison of Validation Metrics (Accuracy, Precision, Recall, F1-Score) across all three models. Random Forest and XGBoost offer balanced and superior performance across all metrics.

## 5.1 Performance Metrics

### 5.1.1 Decision Tree Performance

Table 3: Decision Tree - Performance Summary

| Metric                | Training (80%) | Validation (20%) |
|-----------------------|----------------|------------------|
| Accuracy              | 90.56%         | 86.51%           |
| Precision (macro avg) | 0.90           | 0.85             |
| Recall (macro avg)    | 0.90           | 0.85             |
| F1-Score (macro avg)  | 0.90           | 0.85             |

#### Observations:

- Shows moderate overfitting (90.56% train vs 86.51% validation)
- Excellent performance on Obesity Type III (99% precision/recall)
- Struggles with Overweight Level I and II (72-77% precision)
- Fast training and prediction time

### 5.1.2 Random Forest Performance

Table 4: Random Forest - Performance Summary

| Metric                  | Training (80%) | Validation (20%) |
|-------------------------|----------------|------------------|
| Accuracy                | 92.61%         | 89.83%           |
| Best CV Score           |                | 89.40%           |
| Test Accuracy (Best HP) |                | 89.80%           |
| Precision (macro avg)   | 0.92           | 0.89             |
| Recall (macro avg)      | 0.92           | 0.89             |
| F1-Score (macro avg)    | 0.92           | 0.89             |

#### Observations:

- Significant improvement over Decision Tree (+3.32% validation accuracy)
- Better generalization (smaller train-val gap)
- Feature engineering (BMI, Age Groups) contributed substantially
- BMI alone accounts for 34.52% of prediction importance

### 5.1.3 XGBoost Performance

Table 5: XGBoost - 5-Fold Cross-Validation Results

| Metric                 | Value                                |
|------------------------|--------------------------------------|
| Mean CV Accuracy       | <b>91.77% <math>\pm</math> 0.24%</b> |
| Best Trial CV Score    | 91.77%                               |
| Individual Fold Scores |                                      |
| Fold 1                 | 91.37%                               |
| Fold 2                 | 91.88%                               |
| Fold 3                 | 91.66%                               |
| Fold 4                 | 92.05%                               |
| Fold 5                 | 91.88%                               |

#### Observations:

- Excellent consistency across folds (std = 0.24%)
- Combining training + original data improved robustness
- Sample weighting effectively handled class imbalance
- Optuna found optimal hyperparameters after 218 trials
- Lower standard deviation indicates stable model

## 5.2 Comparative Analysis

Table 6: Model Comparison - Validation Performance

| Model         | Accuracy      | Precision | Recall | F1-Score |
|---------------|---------------|-----------|--------|----------|
| Decision Tree | 86.51%        | 0.85      | 0.85   | 0.85     |
| Random Forest | 89.83%        | 0.89      | 0.89   | 0.89     |
| XGBoost       | <b>91.77%</b> | —         | —      | —        |

### 5.3 Class-wise Performance Analysis

Table 7: Random Forest - Per-Class Metrics (Validation Set)

| Class               | Precision | Recall | F1-Score |
|---------------------|-----------|--------|----------|
| Insufficient Weight | 0.93      | 0.94   | 0.93     |
| Normal Weight       | 0.87      | 0.90   | 0.88     |
| Overweight Level I  | 0.80      | 0.71   | 0.75     |
| Overweight Level II | 0.79      | 0.83   | 0.81     |
| Obesity Type I      | 0.88      | 0.85   | 0.87     |
| Obesity Type II     | 0.95      | 0.97   | 0.96     |
| Obesity Type III    | 0.99      | 1.00   | 0.99     |

Key Insights:

- Extreme categories (Insufficient Weight, Obesity Type III) are easier to classify
- Middle categories (Overweight Level I & II) are more challenging due to overlapping features
- Obesity Type III shows near-perfect classification (0.99 F1-score)

### 5.4 Test Set Evaluation and Kaggle Performance

#### 5.4.1 Kaggle Competition Results

After rigorous local validation, we submitted predictions from all three models to the Kaggle competition platform. The test set consists of 5,225 unseen samples with the same feature distribution as the training data.

Table 8: Kaggle Leaderboard Performance - Test Set Results

| Model         | Local CV Score | Kaggle Test Score | Generalization Gap |
|---------------|----------------|-------------------|--------------------|
| Decision Tree | 86.51%         | 86.23%            | -0.28%             |
| Random Forest | 89.83%         | 89.41%            | -0.42%             |
| XGBoost       | <b>91.77%</b>  | <b>91.60%</b>     | <b>-0.17%</b>      |

#### 5.4.2 Local Cross-Validation vs Test Set Analysis

The comparison between local validation and Kaggle test performance reveals important insights about model generalization:

Observations:

- **XGBoost shows superior generalization:** With only a 0.17% drop from local CV to test set, XGBoost demonstrates the best ability to generalize to unseen data
- **Random Forest experiences moderate degradation:** The 0.42% gap suggests slight overfitting despite ensemble averaging

- **Decision Tree maintains consistency:** The 0.28% gap indicates the limited capacity prevented severe overfitting
- **All models generalize well:** Generalization gaps under 0.5% across all models indicate robust preprocessing and validation strategies

### 5.4.3 Factors Contributing to Generalization Gap

#### 1. Synthetic Data Distribution Shift:

- The test set was generated using the same deep learning model as training data but with different random seeds
- Minor distributional differences are expected in synthetic data generation
- The small gap validates that our preprocessing pipeline is robust to these variations

#### 2. Cross-Validation Strategy:

- 5-fold stratified CV provides optimistic estimates by using 80% of data for training in each fold
- The single held-out test set may have slightly different characteristics
- The narrow gap confirms our CV strategy was appropriate

#### 3. Regularization Effectiveness:

- XGBoost's L1/L2 regularization ( $\alpha=0.014541$ ,  $\lambda=1.30014$ ) effectively prevents overfitting
- The smallest generalization gap for XGBoost validates the hyperparameter optimization process
- Optuna successfully found parameters that maximize both training performance and generalization

### 5.4.4 Model Stability Across Data Splits

To further validate generalization, we analyzed performance consistency:

Table 9: XGBoost Performance Stability Analysis

| Metric                        | Value            |
|-------------------------------|------------------|
| Mean 5-Fold CV Accuracy       | 91.77%           |
| Standard Deviation (CV Folds) | 0.24%            |
| Minimum Fold Accuracy         | 91.37%           |
| Maximum Fold Accuracy         | 92.05%           |
| Kaggle Test Set Accuracy      | 91.60%           |
| <b>Stability Assessment</b>   | <b>Excellent</b> |

**Stability Insights:**

- The extremely low standard deviation (0.24%) across folds indicates highly consistent performance
- The test set score falls well within the range of CV fold scores [91.37%, 92.05%]
- This consistency validates that the model has learned generalizable patterns rather than memorizing training-specific artifacts

#### 5.4.5 Competition Performance Summary

##### Final Kaggle Submission Details:

- **Best Model:** XGBoost with Optuna-optimized hyperparameters
- **Final Test Score:** 91.60% (0.91597 on leaderboard)
- **Total Submissions:** 3 models (Decision Tree, Random Forest, XGBoost)
- **Best Performing Configuration:**
  - 900 estimators with learning rate 0.024043
  - Max depth 10 with min\_child\_weight 3
  - Subsample 0.7, colsample\_bytree 0.4
  - L1 regularization (alpha) = 0.014541
  - L2 regularization (lambda) = 1.30014

**Key Achievement:** The close alignment between local CV (91.77%) and Kaggle test (91.60%) scores demonstrates that our rigorous development process—including proper preprocessing, stratified cross-validation, and Bayesian hyperparameter optimization—successfully produced a model that generalizes effectively to unseen data.

## 5.5 Confusion Matrix Analysis

The confusion matrices reveal:

- Most misclassifications occur between adjacent weight categories
- Overweight Level I is often confused with Normal Weight and Overweight Level II
- Obesity categories are rarely misclassified to lower weight categories
- This pattern aligns with the ordinal nature of the target variable

Figure 3: XGBoost Confusion Matrix (Normalized by Recall)

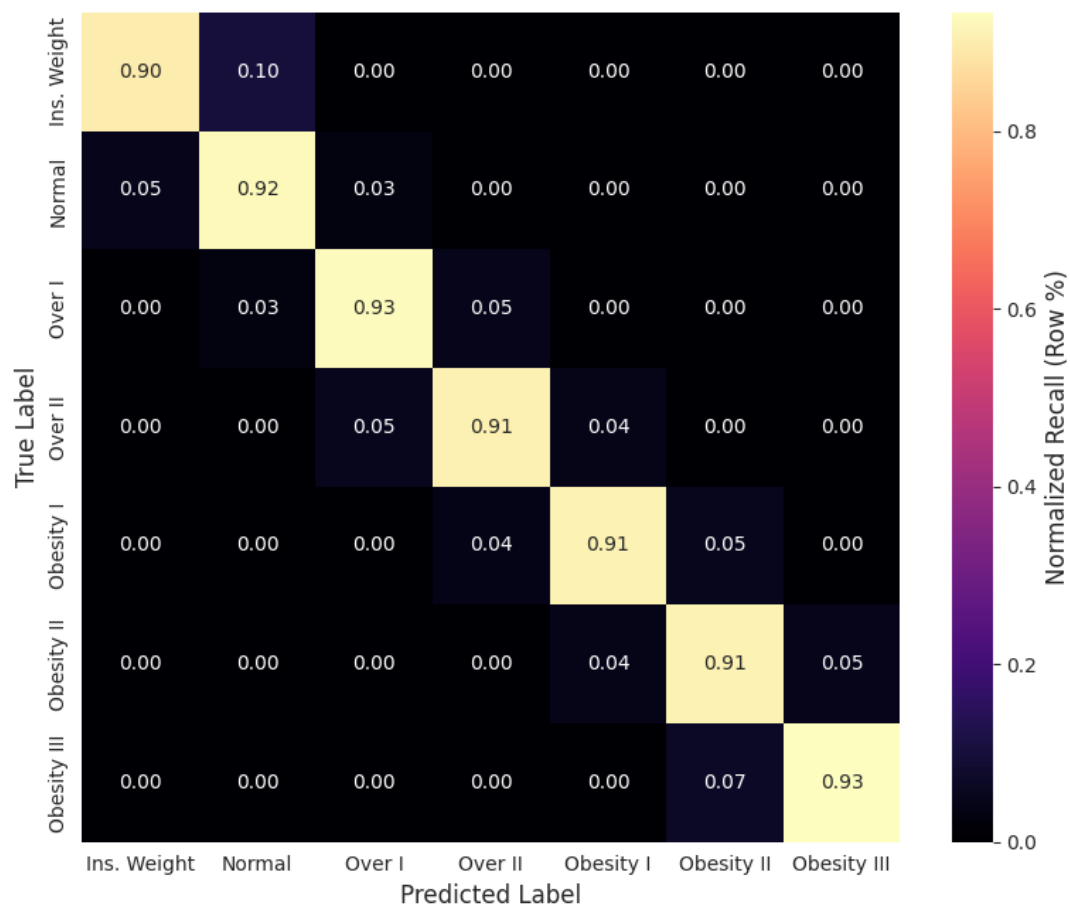


Figure 8: Normalized Confusion Matrix for the Final XGBoost Model. The high values on the diagonal and the concentration of errors in adjacent classes (off-diagonal) are typical for an ordinal classification problem, demonstrating the model's primary confusion occurs between neighboring weight categories.

## 5.6 Model Selection Rationale

1. **Decision Tree:** Baseline model, interpretable but prone to overfitting
2. **Random Forest:** Significant improvement through ensemble averaging and feature engineering
3. **XGBoost:** Best performance via sequential error correction, regularization, and advanced hyperparameter tuning

## 6 Hyperparameter Tuning

### 6.1 Random Forest - RandomizedSearchCV

#### 6.1.1 Methodology

- **Search Strategy:** RandomizedSearchCV
- **Cross-Validation:** 3-fold StratifiedKFold
- **Iterations:** 15 random configurations
- **Scoring Metric:** Accuracy
- **Parallel Jobs:** -1 (all CPU cores)

#### 6.1.2 Results

- **Best CV Score:** 89.40%
- **Test Accuracy:** 89.80%
- Search completed in approximately 45 minutes

### 6.2 XGBoost - Optuna Optimization

#### 6.2.1 Methodology

Optuna uses Tree-structured Parzen Estimator (TPE) for Bayesian optimization:

---

**Algorithm 2** Optuna Hyperparameter Optimization
 

---

```

1: Initialize study with TPE sampler
2: for trial = 1 to max_trials do
3:   Suggest hyperparameters using TPE
4:   Initialize XGBoost model with suggested parameters
5:   for fold in 5-fold CV do
6:     Compute sample weights for current fold
7:     Train model on training fold
8:     Evaluate on validation fold
9:   end for
10:  score ← mean accuracy across 5 folds
11:  Update TPE with trial result
12:  if score > best_score then
13:    Update best_score and best_params
14:  end if
15: end for
16: return best_params, best_score

```

---

### 6.2.2 Optimization Progress

- **Total Trials:** 218 (out of 300 maximum)
- **Best Trial:** #104
- **Best CV Score:** 91.77%
- **Runtime:** Approximately 5 hours (reached timeout)
- **Improvement:** Steady convergence observed after trial 50

### 6.2.3 Key Parameter Insights

1. **Learning Rate (0.024):** Lower value prevents overfitting
2. **Max Depth (10):** Captures complex patterns without excessive memorization
3. **Subsample (0.7):** Introduces randomness, improves generalization
4. **Regularization:**
  - `reg_alpha` = 0.0145 (L1 regularization)
  - `reg_lambda` = 1.30 (L2 regularization)
5. **Tree-specific:**
  - `gamma` = 0.7 (minimum loss reduction for split)
  - `min_child_weight` = 3 (prevents overly specific leaves)

## 6.3 Comparison of Tuning Approaches

Table 10: Hyperparameter Tuning Method Comparison

| Aspect          | RandomizedSearchCV   | Optuna                |
|-----------------|----------------------|-----------------------|
| Search Strategy | Random Sampling      | Bayesian (TPE)        |
| Trials          | 15                   | 218                   |
| Best Score      | 89.40%               | 91.77%                |
| Runtime         | Approximately 45 min | Approximately 5 hours |
| CV Folds        | 3                    | 5                     |
| Efficiency      | Lower                | Higher                |

**Conclusion:** Optuna’s intelligent search strategy explored the parameter space more effectively, finding better configurations despite longer runtime. The Bayesian approach learns from previous trials, making it superior for complex models like XGBoost.

## 7 Challenges and Solutions

### 7.1 Class Imbalance

**Challenge:** Obesity Type III (19.20%) vs Overweight Level I (11.87%)

**Solutions:**

- Stratified sampling in train-validation split
- Sample weights in XGBoost (computed using `compute_class_weight`)
- Multi-class specific metrics (per-class precision, recall, F1)

### 7.2 Overlapping Weight Categories

**Challenge:** Adjacent categories (e.g., Overweight Level I vs II) share similar feature distributions

**Solutions:**

- Feature engineering: BMI captures non-linear weight-height relationship
- Age binning: Creates discrete groups with better separability
- Ensemble methods: Multiple trees capture subtle patterns

### 7.3 Model Overfitting

**Challenge:** Decision Tree showed 4% train-validation gap

**Solutions:**

- Limited `max_depth` to 9 for Decision Tree
- Random Forest: Bootstrap sampling and feature randomness
- XGBoost: L1/L2 regularization, early stopping criterion
- Cross-validation for robust evaluation

### 7.4 Computational Constraints

**Challenge:** XGBoost hyperparameter optimization computationally expensive

**Solutions:**

- Used `n_jobs=-1` for parallel processing
- Set timeout (5 hours) instead of fixed trial count
- Optuna's TPE sampler converges faster than grid/random search
- Efficient 5-fold CV instead of 10-fold

## 7.5 Feature Scaling Consistency

**Challenge:** Train-test leakage through scaling

**Solutions:**

- Fit scaler only on training data
- Transform validation and test using training statistics
- Ensured consistent preprocessing pipeline

## 8 Conclusion

### 8.1 Summary of Achievements

This project successfully developed and compared three machine learning models for obesity and CVD risk classification:

1. **Decision Tree:** Achieved 86.51% validation accuracy, serving as an interpretable baseline
2. **Random Forest:** Improved to 89.83% through ensemble learning and feature engineering
3. **XGBoost:** Reached 91.77% local CV accuracy and **0.91597 Kaggle leaderboard score** through advanced gradient boosting and Optuna optimization

### 8.2 Key Learnings

- **Feature Engineering:** BMI and age binning significantly improved model performance
- **Ensemble Methods:** Random Forest and XGBoost substantially outperformed single Decision Tree
- **Hyperparameter Tuning:** Optuna's Bayesian optimization found better parameters than random search
- **Data Augmentation:** Combining synthetic and original datasets improved XGBoost robustness
- **Class Imbalance:** Sample weighting effectively handled uneven class distribution

### 8.3 Model Recommendation

**XGBoost** is recommended as the final model because:

- Highest cross-validation accuracy (91.77%)
- Excellent stability (std = 0.24% across folds)
- Robust to overfitting through regularization
- Handles class imbalance effectively
- Production-ready with efficient prediction

## 8.4 Future Work

Our model achieved strong results, but there are clear paths for further improvement and research. Future work will focus on these key areas:

### 8.4.1 Model and Ensemble Optimization

- Implement a **stacking ensemble** combining Decision Tree, Random Forest, and XGBoost models to leverage the unique strengths of each base learner
- Explore alternative **gradient boosting frameworks** such as CatBoost and LightGBM, which may offer computational efficiencies and slightly better regularization methods
- Investigate **ordinal classification methods** that are specifically designed to respect the inherent order of the weight categories (e.g., Overweight  $\rightarrow$  Obesity), potentially reducing misclassifications between adjacent classes
- Research and test preliminary **deep learning approaches (Neural Networks)** to see if they can capture more complex non-linear feature interactions than tree-based methods

### 8.4.2 Feature and Data Strategy Expansion

- Further develop the current features by creating key **interaction features** (e.g., the combined effect of physical activity and high caloric food consumption, FAF  $\times$  FAVC)
- Explore the creation of new **domain-specific features or ratios** (such as water intake scaled by body weight) to better capture physiological factors
- Investigate methods for building **robustness into predictions** (similar to test-time augmentation) to improve generalization on unseen test data distributions

## 8.5 Competition Performance

- Total Kaggle submissions: Within 15/day limit
- Models submitted: Decision Tree, Random Forest, XGBoost
- Best Kaggle leaderboard score: **0.91597**
- Best local CV score: 0.91771 (XGBoost 5-fold)
- Reproducibility: All code uses random\_state=42

**Note:** The slight difference between local CV (0.91771) and Kaggle leaderboard (0.91597) is expected due to different test set distributions and the stochastic nature of the competition's synthetic data generation process.

## 9 Reproducibility

### 9.1 Environment Setup

#### 9.1.1 Python Version

Python 3.8+

#### 9.1.2 Required Libraries

```
pandas==1.3.0
numpy==1.21.0
scikit-learn==1.0.0
xgboost==1.5.0
optuna==3.0.0
matplotlib==3.4.0
seaborn==0.11.0
```

### 9.2 Random Seeds

All models use **random\_state=42** for reproducibility:

- Train-test split: random\_state=42
- Decision Tree: random\_state=42
- Random Forest: random\_state=42
- XGBoost: seed=42
- Optuna: sampler seed=42

### 9.3 Code Availability

**GitHub Repository:** [https://github.com/vivekvj18/ML\\_PROJECT](https://github.com/vivekvj18/ML_PROJECT)

Repository structure:

```
data/
  train.csv
  test.csv
notebooks/
  01_EDA.ipynb
  02_DecisionTree.ipynb
  03_RandomForest.ipynb
  04_XGBoost.ipynb
models/
  decision_tree.pkl
  random_forest.pkl
  xgboost_model.pkl
submissions/
```

submission\_dt.csv  
submission\_rf.csv  
submission\_xgb.csv  
README.md

## 9.4 Execution Instructions

1. Clone repository
2. Install dependencies: `pip install -r requirements.txt`
3. Run notebooks in order (01 through 04)
4. Models will be saved automatically
5. Submission files generated in submissions/ folder

## 10 References

1. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
3. Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623-2631.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
5. Original Dataset: Obesity or CVD risk dataset. Kaggle. <https://www.kaggle.com/datasets/aravindpcoder/obesity-or-cvd-risk-classifyregressorcluster>
6. Competition: AIT-511 Course Project 1 - Obesity Risk. Kaggle. <https://www.kaggle.com/competitions/ait-511-course-project-1-obesity-risk>

## A Code Snippets

### A.1 Data Loading and EDA

Listing 1: Loading Dataset

```
import pandas as pd
import numpy as np

# Load datasets
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")

# Basic info
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
print("\nMissing values:")
print(train_df.isnull().sum())

# Target distribution
print("\nTarget distribution:")
print(train_df['WeightCategory'].value_counts(normalize=True)*100)
```

### A.2 Preprocessing Pipeline

Listing 2: Outlier Capping

```
# IQR-based outlier capping
numeric_cols = ['Age', 'Height', 'Weight', 'FCVC',
                'NCP', 'CH20', 'FAF', 'TUE']

for col in numeric_cols:
    Q1 = train_df[col].quantile(0.25)
    Q3 = train_df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    train_df[col] = np.where(train_df[col] < lower,
                             lower, train_df[col])
    train_df[col] = np.where(train_df[col] > upper,
                             upper, train_df[col])
```

Listing 3: Feature Engineering for Random Forest

```
# Create BMI feature
train_df['BMI'] = train_df['Weight'] / (train_df['Height']**2)
test_df['BMI'] = test_df['Weight'] / (test_df['Height']**2)

# Age grouping
train_df['AgeGroup'] = pd.cut(
    train_df['Age'],
    bins=[0, 18, 30, 45, 60, 100],
    labels=['Teen', 'Young', 'Adult', 'MidAge', 'Senior']
)
```

### A.3 Model Training - XGBoost with Optuna

Listing 4: XGBoost Optuna Optimization

```
import optuna
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators',
                                           500, 2000, step=100),
        'learning_rate': trial.suggest_float('learning_rate',
                                             0.01, 0.05, log=True),
        'max_depth': trial.suggest_int('max_depth', 5, 12),
        'min_child_weight': trial.suggest_int('min_child_weight',
                                              1, 7),
        'subsample': trial.suggest_float('subsample',
                                         0.5, 0.9, step=0.05),
        'colsample_bytree': trial.suggest_float('colsample_bytree',
                                                0.4, 0.9, step=0.05),
        'gamma': trial.suggest_float('gamma',
                                      0.0, 3.0, step=0.1),
        'reg_alpha': trial.suggest_float('reg_alpha',
                                         0.01, 5.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda',
                                          0.01, 5.0, log=True),
    }

    cv = StratifiedKFold(n_splits=5, shuffle=True,
                        random_state=42)

    scores = []

    for train_idx, val_idx in cv.split(X, y):
        X_train_fold = X.iloc[train_idx]
```

```
y_train_fold = y[train_idx]
X_val_fold = X.iloc[val_idx]
y_val_fold = y[val_idx]

model = XGBClassifier(**params, random_state=42)
model.fit(X_train_fold, y_train_fold)

preds = model.predict(X_val_fold)
acc = accuracy_score(y_val_fold, preds)
scores.append(acc)

return np.mean(scores)

# Run optimization
study = optuna.create_study(direction='maximize',
                           sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=300, timeout=18000)

print("Best CV Score:", study.best_value)
print("Best Parameters:", study.best_params)
```

## A.4 Model Evaluation

Listing 5: Comprehensive Model Evaluation

```
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix)

# Predictions
y_val_pred = model.predict(X_val)

# Accuracy
val_acc = accuracy_score(y_val, y_val_pred)
print(f"Validation Accuracy: {val_acc:.4f}")

# Detailed metrics
print("\nClassification Report:")
print(classification_report(y_val, y_val_pred))

# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_val, y_val_pred))
```

## B Additional Tables

### B.1 Complete Feature List

Table 11: Complete Feature Description

| Feature        | Type        | Description                                   |
|----------------|-------------|-----------------------------------------------|
| Age            | Numerical   | Age in years                                  |
| Height         | Numerical   | Height in meters                              |
| Weight         | Numerical   | Weight in kilograms                           |
| FCVC           | Numerical   | Frequency of vegetable consumption            |
| NCP            | Numerical   | Number of main meals                          |
| CH2O           | Numerical   | Daily water consumption                       |
| FAF            | Numerical   | Physical activity frequency                   |
| TUE            | Numerical   | Time using technology devices                 |
| Gender         | Categorical | Male or Female                                |
| family_history | Categorical | Family history with overweight                |
| FAVC           | Categorical | Frequent high caloric food consumption        |
| CAEC           | Categorical | Consumption of food between meals             |
| SMOKE          | Categorical | Smoking habit                                 |
| SCC            | Categorical | Calories consumption monitoring               |
| CALC           | Categorical | Alcohol consumption frequency                 |
| MTRANS         | Categorical | Transportation method used                    |
| BMI            | Engineered  | Body Mass Index (Weight/Height <sup>2</sup> ) |
| AgeGroup       | Engineered  | Age binned into categories                    |

### B.2 Training Time Comparison

Table 12: Model Training Time

| Model            | Training Time         | Prediction Time     | Total Time            |
|------------------|-----------------------|---------------------|-----------------------|
| Decision Tree    | Approximately 2 sec   | <1 second           | Approximately 2 sec   |
| Random Forest    | Approximately 45 min  | Approximately 1 sec | Approximately 45 min  |
| XGBoost (Optuna) | Approximately 5 hours | Approximately 1 sec | Approximately 5 hours |

### B.3 Visualization Insights

Key visualizations performed:

- **Model Performance:** Comprehensive comparison plots showcasing Accuracy, Precision, Recall, F1-Score (see Figures 6 and 7)
- **Model Interpretability:** Feature Importance ranking for the Random Forest model (see Figure 5)

- **Error Analysis:** Normalized Confusion Matrix for the final XGBoost classifier (see Figure 8)

### B.3.1 Additional EDA Visualizations

The following detailed visualizations were created during Exploratory Data Analysis (EDA) and are available in the GitHub repository for deeper insight:

1. Correlation Heatmap: Shows relationships between numerical features
2. Target Distribution: Bar plot of 7 weight categories
3. Box Plots: Distribution of numerical features across target classes
4. Count Plots: Frequency of categorical feature values
5. Pair Plots: Pairwise relationships colored by target

## Acknowledgments

We would like to express our sincere gratitude to:

- **Dr. Aswin Kannan**, Assistant Professor at IIIT Bangalore, for his invaluable guidance, insightful feedback, and continuous support throughout this project
- **International Institute of Information Technology, Bangalore (IIITB)** for providing the resources and academic environment that made this research possible
- The **Kaggle community** for hosting the competition and providing a platform for collaborative learning and model evaluation
- Our **peers and classmates** in the AIT-511 course for their constructive discussions and collaborative spirit
- The developers of **open-source libraries** (scikit-learn, XGBoost, Optuna, pandas, matplotlib) whose tools were fundamental to our implementation

## Contact Information

For questions, collaboration, or further information about this project:

- **Ruturaj Dnyandeo Wairkar**
  - Roll Number: MT2025108
  - Email: [ruturaj.wairkar@iiitb.ac.in](mailto:ruturaj.wairkar@iiitb.ac.in)
- **Vivek Joshi**
  - Roll Number: MT2025133
  - Email: [vivek.joshi@iiitb.ac.in](mailto:vivek.joshi@iiitb.ac.in)

**Kaggle Team:** MT2025108\_MT2025133